FitzCLIM

A Common Lisp Implementation of the CLIM II Protocols and the CLIM 2.2 Revisions

https://github.com/cgore/FitzCLIM/

Christopher Mark Gore
http://www.cgore.com
cgore@cgore.com

Friday, October 23rd, AD 2026

Contents

Copyright **vii**

Introduction **ix**

1 The `FitzCLIM/geometry` Package **1**
1.1 Variables 1
1.1.1 The `+nowhere+` Variable 1
1.1.2 The `+everywhere+` Variable 1
1.1.3 The `+identity-transformation+` Variable 1
1.2 Functions 2
1.2.1 The `regionp` Function 2
1.2.2 The `nowhere-p` Function 2
1.2.3 The `everywhere-p` Function 2
1.2.4 The `region-set-p` Function 2
1.2.5 The `pointp` Function 2
1.2.6 The `make-point` Function 2
1.2.7 The `point-x` Function 2
1.2.8 The `point-y` Function 3
1.2.9 The `point-position` Function 3
1.2.10 The `rectanglep` Function 3
1.2.11 The `make-rectangle` Function 3
1.2.12 The `make-rectangle*` Function 3
1.2.13 The `rectangle-min-x` Function 3
1.2.14 The `rectangle-min-y` Function 4
1.2.15 The `rectangle-max-x` Function 4
1.2.16 The `rectangle-max-y` Function 4
1.2.17 The `rectangle-edges*` Function 4
1.2.18 The `rectangle-width` Function 4
1.2.19 The `rectangle-height` Function 4
1.2.20 The `rectangle-size` Function 4
1.2.21 The `rectangle-intersection-or-nil` Function . 4
1.2.22 The `rectangle-difference-list` Function 5
1.2.23 The `rectangles-touch-horizontally-p` Function 5
1.2.24 The `rectangles-touch-vertically-p` Function . 5

1.2.25 The `join-two-rectangles` Function 5
1.2.26 The `polygonp` Function 5
1.2.27 The `make-polygon` Function 6
1.2.28 The `make-polygon*` Function 6
1.2.29 The `polygon-points` Function 6
1.2.30 The `polygon-convex-p` Function 6
1.2.31 The `map-polygon-edges` Function 6
1.2.32 The `transformationp` Function 7
1.2.33 The `make-transformation` Function 7
1.2.34 The `make-translation-transformation` Function 7
1.2.35 The `make-scaling-transformation` Function . . . 7
1.2.36 The `make-rotation-transformation` Function . . 8
1.2.37 The `compose-transformations` Function 8
1.2.38 The `invert-transformation` Function 8
1.2.39 The `transform-position` Function 8
1.2.40 The `untransform-position` Function 9
1.2.41 The `transformation-rectilinear-p` Function . . 9
1.2.42 The `bounding-rectangle` Function 9
1.2.43 The `bounding-rectangle-min-x` Function 9
1.2.44 The `bounding-rectangle-min-y` Function 10
1.2.45 The `bounding-rectangle-max-x` Function 10
1.2.46 The `bounding-rectangle-max-y` Function 10
1.2.47 The `bounding-rectangle-width` Function 10
1.2.48 The `bounding-rectangle-height` Function 10
1.2.49 The `bounding-rectangle-size` Function 10
1.3 Generics 10
1.3.1 The `region-contains-position-p` Generic 10
1.3.2 The `region-contains-region-p` Generic 11
1.3.3 The `region-equal` Generic 11
1.3.4 The `region-intersects-region-p` Generic 11
1.3.5 The `region-union` Generic 12
1.3.6 The `region-intersection` Generic 12
1.3.7 The `region-difference` Generic 12
1.3.8 The `bounding-rectangle*` Generic 13
1.3.9 The `transform-region` Generic 13
1.4 Types 14
1.4.1 The `region` Type 14
1.4.2 The `nowhere` Type 14
1.4.3 The `everywhere` Type 14
1.4.4 The `point` Type 14
1.4.5 The `rectangle` Type 14
1.4.6 The `polygon` Type 14
1.4.7 The `region-set` Type 15
1.4.8 The `standard-region-difference` Type 15
1.4.9 The `transformation` Type 15
1.5 Conditions 15

1.5.1 The `singular-transformation` Condition 15
1.5.2 The `bounding-rectangle-of-nowhere` Condition . 15

Copyright

Copyright © 2026, Christopher Mark Gore,
Soli Deo Gloria,
All rights reserved.
22 Forest Glade Court, Saint Charles, Missouri 63304 USA.
Web: `http://www.cgore.com`
Email: cgore@cgore.com
Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Christopher Mark Gore nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Introduction

FitzCLIM is a Common Lisp implementation of the CLIM II protocols [4] and of the CLIM 2.2 revisions. The 2.2 revisions are recorded in Franz's CLIM 2.2 User Guide [1], not in a second formal specification. That guide is copyrighted. This manual does not copy it. FitzCLIM is aimed at SBCL.

The frame manager realizes the standard gadgets as native controls. On Windows those are Windows controls. On macOS those are Cocoa controls. Layout positions them through the sheet protocol. A native control does not hold output records. Its items may still be Lisp objects, and a selection may run a command.

A presentation type may have an abbreviation. The abbreviation expands to another presentation type, and a test or a presentation uses that type. An application pane is drawn by FitzCLIM. It records output and holds presentations. A command loop reads commands, from typing or from a gesture on a presentation, and runs them. After a command, panes that have a display function are drawn again.

`define-application-frame` builds the frame class and its command table. `:command-definer` names the macro that adds a command to that table. The default name is `define-`*`frame`*`-command`. `:menu-bar` chooses the command table shown as the menu bar, or `nil` for none. `:disabled-commands` names commands that start disabled. `command-enabled` turns one on or off. `:top-level` names the function that runs the command loop. A command argument may name a `:gesture`. That gesture on a presentation of the argument's type runs the command. The other arguments take their `:default` values. A command line that ends early uses those defaults too. An argument a gesture did not supply is marked unsupplied. Running that command fills the mark from the default, or asks for the value when there is none. `:keystroke` on the command is a character that runs it from the menu. `:pane` supplies one pane. `:panes` and `:layouts` supply named panes and more than one arrangement. An `:interactor` pane reads a typed command line. The command loop writes its prompt there and reads the next line when no gesture has produced a command. `read-command` reads that line through the command parser. The parser, the unparser, and the partial-command parser can be re-

placed for one call.

A 3D pane is a native OpenGL child window, a sibling of the native controls. Pointer events at its edge become a ray. A hit returns a Lisp object and enters the same command and translator machinery as a 2D presentation. The OpenGL pane owns its redraw. FitzCLIM may ask that pane to repaint. The pane does not record its scene as output records. A label in the scene is drawn and picked there.

The 2D graphics protocol stays two-dimensional. Regions, transforms, clipping, formatted output, and output recording are a plane. They stay out of the scene graph.

FitzCLIM was begun so that EDEFB [2], the Elite Dangerous Electronic Flight Bag, can have a CLIM. The toolkit is general. Other programs will use it too. Elite Dangerous stays in the EDEFB repository.

This is a clean-room implementation. It is not derived from McCLIM, from Franz CLIM, or from LispWorks CLIM. The specification may be consulted. Its text stays out of this tree.

Tests use Σ [3], in particular the `behavior` and `should` forms in `sigma/behave`.

The frame managers target Windows and macOS. Geometry, ink, sheets, commands, frames, events, presentations, output recording, gadgets, pane layout, and the frame managers are under `source/`. An application pane records what it draws and finds the presentation under a pointer. A frame can stack its panes. A gadget holds a value or runs a callback. On Windows the frame manager realizes that frame as a window and those gadgets as native controls. Pointer and keyboard input on an application pane become CLIM events, and a paint replays the recorded output. On macOS the frame manager realizes the frame as a Cocoa window and the gadgets as Cocoa controls. Pointer and keyboard input become CLIM events, and a menu of the frame's commands is installed in the menu bar.

You can download the library from GitHub at:
`https://github.com/cgore/FitzCLIM`

Getting Lisp

Before using this library you need a working Lisp. I use and recommend SBCL, Steel Bank Common Lisp, which is available at:
`http://www.sbcl.org`

Getting EMACS and SLIME

The best way to interact with any Common Lisp is via SLIME, the Superior Lisp Interaction Mode for EMACS. It is available at:
`https://common-lisp.net/project/slime/`

Chapter 1

The FitzCLIM/geometry Package

The FitzCLIM/geometry package is the geometry protocol. It provides points, rectangles, polygons, affine transformations, and the region operations on them: containment, union, intersection, difference, equality, and bounding rectangles. The protocol stays two-dimensional. Regions, transforms, and clipping are a plane, and they stay out of the scene graph of the 3D pane.

1.1 Variables

1.1.1 The +nowhere+ Variable

The +nowhere+ variable holds the empty region, an instance of the nowhere class. It contains no positions.

1.1.2 The +everywhere+ Variable

The +everywhere+ variable holds the region that contains every position, an instance of the everywhere class.

1.1.3 The +identity-transformation+ Variable

The +identity-transformation+ variable holds the transformation that leaves every position where it is.

1.2 Functions

1.2.1 The regionp Function

The regionp function returns true if object is a region. Points, rectangles, polygons, and region sets are regions.

1.2.2 The nowhere-p Function

The nowhere-p function returns true if object is the empty region.

1.2.3 The everywhere-p Function

The everywhere-p function returns true if object is the whole plane.

1.2.4 The region-set-p Function

The region-set-p function returns true if object is a region set.

1.2.5 The pointp Function

The pointp function returns true if object is a point.

1.2.6 The make-point Function

The make-point function returns a point at the coordinates x and y, which must be real.

Syntax

```
(make-point x y)
```

Examples

```
(let ((point (make-point 3 4)))
  (pointp point)   ; Returns T
  (point-x point)  ; Returns 3
  (point-y point)) ; Returns 4
```

1.2.7 The point-x Function

The point-x function returns the x coordinate of point.

1.2.8 The `point-y` Function

The `point-y` function returns the y coordinate of `point`.

1.2.9 The `point-position` Function

The `point-position` function returns the x and y coordinates of `point` as two values.

1.2.10 The `rectanglep` Function

The `rectanglep` function returns true if `object` is a rectangle.

1.2.11 The `make-rectangle` Function

The `make-rectangle` function returns the axis-aligned rectangle with opposite corners `point-1` and `point-2`.

Syntax

`(make-rectangle` *point-1 point-2*`)`

1.2.12 The `make-rectangle*` Function

The `make-rectangle*` function returns the axis-aligned rectangle with corners `(x1, y1)` and `(x2, y2)`. The corner order does not matter.

Syntax

`(make-rectangle*` *x1 y1 x2 y2*`)`

Examples

```
(let ((rectangle (make-rectangle* 4 1 0 3)))
  (rectangle-min-x rectangle)   ; Returns 0
  (rectangle-min-y rectangle)   ; Returns 1
  (rectangle-max-x rectangle)   ; Returns 4
  (rectangle-max-y rectangle)   ; Returns 3
  (rectangle-width rectangle)   ; Returns 4
  (rectangle-height rectangle)) ; Returns 2
```

1.2.13 The `rectangle-min-x` Function

The `rectangle-min-x` function returns the left edge of `rectangle`.

1.2.14 The rectangle-min-y Function

The rectangle-min-y function returns the top edge of rectangle.

1.2.15 The rectangle-max-x Function

The rectangle-max-x function returns the right edge of rectangle.

1.2.16 The rectangle-max-y Function

The rectangle-max-y function returns the bottom edge of rectangle.

1.2.17 The rectangle-edges* Function

The rectangle-edges* function returns the left, top, right, and bottom edges of rectangle as four values.

1.2.18 The rectangle-width Function

The rectangle-width function returns the width of rectangle.

1.2.19 The rectangle-height Function

The rectangle-height function returns the height of rectangle.

1.2.20 The rectangle-size Function

The rectangle-size function returns the width and height of rectangle as two values.

1.2.21 The rectangle-intersection-or-nil Function

The rectangle-intersection-or-nil function returns the intersection of the two rectangles, or nil when they miss. Touching at an edge or a corner counts as an intersection.

Syntax

(rectangle-intersection-or-nil *rectangle-1 rectangle-2*)

Examples

```
(let ((one (make-rectangle* 0 0 4 4))
      (two (make-rectangle* 2 2 6 6)))
  (rectangle-equal (rectangle-intersection-or-nil one two)
                   (make-rectangle* 2 2 4 4))) ; Returns T
```

1.2.22 The `rectangle-difference-list` Function

The `rectangle-difference-list` function returns the parts of `rectangle-1` that lie outside `rectangle-2`, as a list of rectangles. An empty list means `rectangle-2` covers `rectangle-1`. When the two rectangles miss, the list holds `rectangle-1` itself.

Syntax

`(rectangle-difference-list` *`rectangle-1 rectangle-2`*`)`

Examples

```
(let* ((whole (make-rectangle* 0 0 10 10))
       (hole (make-rectangle* 2 2 8 8))
       (pieces (rectangle-difference-list whole hole)))
  (length pieces))              ; Returns 4
(rectangle-difference-list hole whole) ; Returns NIL
```

1.2.23 The `rectangles-touch-horizontally-p` Function

The `rectangles-touch-horizontally-p` function returns true when the two rectangles share a vertical edge and the same vertical span, so their union is one rectangle.

1.2.24 The `rectangles-touch-vertically-p` Function

The `rectangles-touch-vertically-p` function returns true when the two rectangles share a horizontal edge and the same horizontal span, so their union is one rectangle.

1.2.25 The `join-two-rectangles` Function

The `join-two-rectangles` function returns one rectangle covering both arguments. The caller has already decided that the union is itself a rectangle.

1.2.26 The `polygonp` Function

The `polygonp` function returns true if `object` is a polygon.

1.2.27 The make-polygon Function

The make-polygon function returns a polygon through points, which must hold at least three points. The :convex-p keyword marks the polygon convex; containment of another region then tests the corners only.

Syntax

(make-polygon *points* &key *convex-p*)

Examples

```
(let ((square (make-polygon* 0 0 4 0 4 3 0 3)))
  (polygonp square)                    ; Returns T
  (length (polygon-points square))     ; Returns 4
  (region-contains-position-p square 1 1)   ; Returns T
  (region-contains-position-p square 0 0)   ; Returns T
  (region-contains-position-p square 4 3)   ; Returns T
  (region-contains-position-p square 5 1))  ; Returns NIL
```

1.2.28 The make-polygon* Function

The make-polygon* function returns a polygon from the flat coordinate list coordinates, which must pair x and y.

Syntax

(make-polygon* &rest *coordinates*)

1.2.29 The polygon-points Function

The polygon-points function returns the vertices of polygon, in order. The first vertex is not repeated at the end.

1.2.30 The polygon-convex-p Function

The polygon-convex-p function returns true when polygon is marked convex.

1.2.31 The map-polygon-edges Function

The map-polygon-edges function calls function on each edge of polygon as four coordinates, x1 y1 x2 y2. The last edge closes back to the first vertex.

Syntax

```
(map-polygon-edges polygon function)
```

1.2.32 The `transformationp` Function

The `transformationp` function returns true if `object` is a transformation.

1.2.33 The `make-transformation` Function

The `make-transformation` function returns the affine transformation with the given matrix and translation. It maps a position by $x' = \mathtt{mxx}\,x + \mathtt{mxy}\,y + \mathtt{tx}$ and $y' = \mathtt{myx}\,x + \mathtt{myy}\,y + \mathtt{ty}$.

Syntax

```
(make-transformation mxx mxy myx myy tx ty)
```

1.2.34 The `make-translation-transformation` Function

The `make-translation-transformation` function returns a transformation that adds `dx` and `dy` to a position.

Syntax

```
(make-translation-transformation dx dy)
```

Examples

```
(multiple-value-bind (x y)
    (transform-position (make-translation-transformation 10 -2) 1 1)
  (list x y)) ; Returns (11 -1)
```

1.2.35 The `make-scaling-transformation` Function

The `make-scaling-transformation` function returns a transformation that scales by `sx` and `sy` about `origin`, which defaults to `(0, 0)`.

Syntax

```
(make-scaling-transformation sx sy &optional origin)
```

Examples

```
(multiple-value-bind (x y)
    (transform-position (make-scaling-transformation 2 3) 4 5)
  (list x y)) ; Returns (8 15)
```

1.2.36 The `make-rotation-transformation` Function

The `make-rotation-transformation` function returns a transformation that rotates counterclockwise by `angle` radians about `origin`, which defaults to `(0, 0)`.

Syntax

`(make-rotation-transformation` *angle* `&optional` *origin*`)`

1.2.37 The `compose-transformations` Function

The `compose-transformations` function returns the transformation that applies `transformation-2` first and `transformation-1` second.

Syntax

`(compose-transformations` *transformation-1 transformation-2*`)`

1.2.38 The `invert-transformation` Function

The `invert-transformation` function returns the inverse of `transformation`. It signals `SINGULAR-TRANSFORMATION` when the transformation has no inverse.

1.2.39 The `transform-position` Function

The `transform-position` function returns the image of `(x, y)` under `transformation`, as two values.

Syntax

`(transform-position` *transformation x y*`)`

Examples

```
(multiple-value-bind (x y) (transform-position +identity-transformation+ 3 4)
  (list x y)) ; Returns (3 4)
```

1.2.40 The `untransform-position` Function

The `untransform-position` function returns the position that `transformation` maps to `(x, y)`.

Syntax

`(untransform-position` *`transformation x y`*`)`

Examples

```
(multiple-value-bind (x y)
    (untransform-position (make-translation-transformation 5 7) 8 9)
  (list x y)) ; Returns (3 2)
```

1.2.41 The `transformation-rectilinear-p` Function

The `transformation-rectilinear-p` function returns true when `transformation` maps axis-aligned rectangles to axis-aligned rectangles. A rotation of a quarter turn does; a shear does not.

Examples

```
(transformation-rectilinear-p +identity-transformation+) ; Returns T
(transformation-rectilinear-p (make-transformation 0 -1 1 0 0 0)) ; Returns T
(transformation-rectilinear-p (make-transformation 1 -1 1 1 0 0)) ; Returns NIL
```

1.2.42 The `bounding-rectangle` Function

The `bounding-rectangle` function returns the smallest axis-aligned rectangle that covers `region`. The empty region and the whole plane have no such rectangle, and the function signals `BOUNDING-RECTANGLE-OF-NOWHERE` for them.

Examples

```
(bounding-rectangle (make-polygon* 0 1 3 4 1 2))
  ; Returns the rectangle from (0, 1) to (3, 4)
```

1.2.43 The `bounding-rectangle-min-x` Function

The `bounding-rectangle-min-x` function returns the left edge of the bounding rectangle of `region`.

1.2.44 The bounding-rectangle-min-y Function

The bounding-rectangle-min-y function returns the top edge of the bounding rectangle of region.

1.2.45 The bounding-rectangle-max-x Function

The bounding-rectangle-max-x function returns the right edge of the bounding rectangle of region.

1.2.46 The bounding-rectangle-max-y Function

The bounding-rectangle-max-y function returns the bottom edge of the bounding rectangle of region.

1.2.47 The bounding-rectangle-width Function

The bounding-rectangle-width function returns the width of the bounding rectangle of region.

1.2.48 The bounding-rectangle-height Function

The bounding-rectangle-height function returns the height of the bounding rectangle of region.

1.2.49 The bounding-rectangle-size Function

The bounding-rectangle-size function returns the width and height of the bounding rectangle of region as two values.

1.3 Generics

1.3.1 The region-contains-position-p Generic

The region-contains-position-p generic returns true when the position (x, y) lies in region. Boundaries count as inside.

Syntax

(region-contains-position-p *region x y*)

Examples

```
(region-contains-position-p (make-rectangle* 0 1 4 3) 0 1) ; Returns T
(region-contains-position-p (make-rectangle* 0 1 4 3) 2 2) ; Returns T
(region-contains-position-p (make-rectangle* 0 1 4 3) -1 2) ; Returns NIL
```

1.3.2 The `region-contains-region-p` Generic

The `region-contains-region-p` generic returns true when every point of `region2` lies in `region1`.

Syntax

`(region-contains-region-p` *region1 region2*`)`

Examples

```
(region-contains-region-p +everywhere+ +nowhere+) ; Returns T
(region-contains-region-p +nowhere+ +everywhere+) ; Returns NIL
```

1.3.3 The `region-equal` Generic

The `region-equal` generic returns true when `region1` and `region2` contain the same points.

Syntax

`(region-equal` *region1 region2*`)`

Examples

```
(region-equal (make-point 3 4) (make-point 3 4)) ; Returns T
(region-equal (make-point 3 4) (make-point 3 5)) ; Returns NIL
```

1.3.4 The `region-intersects-region-p` Generic

The `region-intersects-region-p` generic returns true when `region1` and `region2` have at least one point in common.

Syntax

`(region-intersects-region-p` *region1 region2*`)`

Examples

```
(region-intersects-region-p (make-rectangle* 0 0 2 2)
                            (make-rectangle* 2 0 5 2)) ; Returns T
(region-intersects-region-p +everywhere+ +nowhere+) ; Returns NIL
```

1.3.5 The `region-union` Generic

The `region-union` generic returns the region that contains a point exactly when `region` or one of `regions` contains it. Rectangles whose union is a single rectangle are joined; points inside other members are dropped.

Syntax

`(region-union` *region* `&rest` *regions*`)`

Examples

```
(let ((left (make-rectangle* 0 0 2 2))
      (right (make-rectangle* 2 0 5 2))
      (joined (region-union left right)))
  (list (rectanglep joined) (rectangle-width joined))) ; Returns (T 5)
```

1.3.6 The `region-intersection` Generic

The `region-intersection` generic returns the region of points that lie in `region` and in every region in `regions`.

Syntax

`(region-intersection` *region* `&rest` *regions*`)`

Examples

```
(region-equal (region-intersection (make-rectangle* 0 0 4 4)
                                   (make-rectangle* 2 2 6 6))
              (make-rectangle* 2 2 4 4)) ; Returns T
(region-equal +nowhere+ (region-intersection +nowhere+ +everywhere+)) ; Returns T
```

1.3.7 The `region-difference` Generic

The `region-difference` generic returns the points of `region1` that do not lie in `region2`.

Syntax

(region-difference *region1 region2*)

Examples

```
(let ((rest (region-difference (make-rectangle* 0 0 4 2)
                               (make-rectangle* 1 0 2 2))))
  (list (region-set-p rest)
        (region-contains-position-p rest 0.5 1)
        (region-contains-position-p rest 1.5 1)
        (region-contains-position-p rest 3 1)))
  ; Returns (T T NIL T)
(region-equal +nowhere+ (region-difference +everywhere+ +everywhere+)) ; Returns T
```

1.3.8 The `bounding-rectangle*` Generic

The `bounding-rectangle*` generic returns the left, top, right, and bottom edges of the smallest axis-aligned rectangle that covers `region`, as four values. It signals `BOUNDING-RECTANGLE-OF-NOWHERE` for the empty region and the whole plane.

Syntax

(bounding-rectangle* *region*)

Examples

```
(multiple-value-bind (left top right bottom)
    (bounding-rectangle* (make-polygon* 0 1 3 4 1 2))
  (list left top right bottom)) ; Returns (0 1 3 4)
```

1.3.9 The `transform-region` Generic

The `transform-region` generic returns the image of `region` under `transformation`. A rectangle stays a rectangle when the transformation is rectilinear, and becomes a convex polygon when it is not.

Syntax

(transform-region *transformation region*)

Examples

```
(let ((shift (make-translation-transformation 10 0)))
  (region-equal (transform-region shift (make-rectangle* 0 0 4 3))
                (make-rectangle* 10 0 14 3))) ; Returns T
(let* ((shear (make-transformation 1 0.5 0 1 0 0))
       (image (transform-region shear (make-rectangle* 0 0 2 2))))
  (list (polygonp image) (polygon-convex-p image))) ; Returns (T T)
```

1.4 Types

1.4.1 The `region` Type

The `region` type is the abstract base of the geometry protocol. A region is a set of points in the plane. The subclasses are `nowhere`, `everywhere`, `point`, `rectangle`, `polygon`, and `region-set`.

1.4.2 The `nowhere` Type

The `nowhere` type is the empty region. It contains no positions.

1.4.3 The `everywhere` Type

The `everywhere` type is the whole plane.

1.4.4 The `point` Type

The `point` type is a single position in the plane, stored as two real coordinates in `standard-point`.

1.4.5 The `rectangle` Type

The `rectangle` type is an axis-aligned rectangle. Top is the smaller y coordinate, and bottom is the larger one. The boundary is inside the rectangle.

1.4.6 The `polygon` Type

The `polygon` type is a filled polygon. The boundary is inside. `standard-polygon` stores the vertices as a list of points and keeps a convexity mark.

1.4.7 The `region-set` Type

The `region-set` type is a region made of member regions. A position is inside the set when it is inside some member. `standard-region-set` stores the members as a list, in no particular order.

1.4.8 The `standard-region-difference` Type

The `standard-region-difference` type is the region of points that lie in one region and not in another, stored as the minuend and the subtrahend. A difference of rectangles that splits into rectangles is stored as those rectangles instead.

1.4.9 The `transformation` Type

The `transformation` type is a mapping of positions to positions. `standard-transformation` is the affine form: $x' = \mathtt{mxx}\,x + \mathtt{mxy}\,y + \mathtt{tx}$ and $y' = \mathtt{myx}\,x + \mathtt{myy}\,y + \mathtt{ty}$.

1.5 Conditions

1.5.1 The `singular-transformation` Condition

The `singular-transformation` condition signals that a transformation has no inverse. Its linear part collapses the plane onto a line or a point.

1.5.2 The `bounding-rectangle-of-nowhere` Condition

The `bounding-rectangle-of-nowhere` condition signals that a bounding rectangle was requested for the empty region or the whole plane, which have no edges.

Bibliography

[1] FRANZ INC. *CLIM 2.2 User Guide*, 2.2.2 ed., 2005. September 2005. Copyright Franz Inc. Consult; do not copy.

[2] GORE, C. M. *EDEFB: the Elite Dangerous Electronic Flight Bag*, 2026. `https://github.com/cgore/edefb`.

[3] GORE, C. M. *Σ: A Library for ANSI Common Lisp*, 2026. `https://github.com/cgore/sigma`.

[4] MCKAY, S., AND YORK, W. *Common Lisp Interface Manager: CLIM II Specification*, 1994.